Komunikacja z czujnikiem prądu TLI4971

Michał Czarnocki 151112
Mikołaj Aleksandrzak 147458

25.09.2025

Contents

1 Wstęp **2**

2 Wymagane urządzenia, programy i dokumenty **2**
2.1 Hardware 2
2.2 Software 2
2.3 Dokumentacja urządzeń 2

3 Układ elektryczny **2**
3.1 Wstęp teoretyczny 2
3.2 Układ elektryczny 3

4 Protokół komunikacji **3**
4.1 Wstęp 3
4.2 Wysyłanie jednego bita 4
4.3 Komendy 5
4.4 Aktywowanie interfejsu cyfrowego 6
4.5 Zmienianie pamięci trwałej 6
4.5.1 Sekwencja zapisania pamięci trwałej 6
4.5.2 Ustawienia czujnika 7
4.5.3 Obliczanie CRC 7
4.5.4 Zastosowanie 7

5 Program **8**
5.1 Wstęp 8
5.2 Działanie programu 8
5.3 Tryby pracy programu 10

6 Problem ze stabilizatorem napięcia **11**

1 Wstęp

Dokumentacja omawia zrealizowany projekt modyfikujący pamięć trwałą czujnika TLI4971.

W dokumentacji omówiony został układ elektryczny zrealizowanego projektu oraz program i jak z niego korzystać.

2 Wymagane urządzenia, programy i dokumenty

2.1 Hardware

Do rozwiązania problemu komunikacji z płytką TLI4971 zastosowano:

- Płytkę STM32 Nucleo-144 F746ZG,
- Stabilizator napięcia TC1014,

2.2 Software

- Do zaprogramowania płytki Nucleo użyto STM32CubeIDE 1.18.0, dostępny na stronie `https://www.st.com/en/development-tools/stm32cubeide.html`.
- Do przeczytania pamięci stałej płytki TLI4971 wykorzystano program XENSIV current sensor M2SGO Evaluation App: `https://softwaretools.infineon.com/tools/com.ifx.tb.tool.tlx4971currentsensorsevaluationapp`

2.3 Dokumentacja urządzeń

- Kluczowym dokumentem, na którym opiera się cały projekt jest **dokumentacja programowania czujnika TLI4971**: `https://www.infineon.com/gated/infineon-tli4971-progguide-usermanual-en_97411bb3-50d1-4e5b-9e1f-7807b910bbd6`
- Stabilizator napięcia **TC1014** służący do zasilenia czujnika: `https://www.mouser.pl/datasheet/3/282/1/-1185-50-mA-100-mA-and-150-mA-CMOS-LDOs-with-Shutdown-and-Reference-Bypass-Data-Sheet-DS20001335.pdf`

3 Układ elektryczny

3.1 Wstęp teoretyczny

Komunikacja w obie strony po jednym połączeniu możliwe jest przez zastosowanie open-drain. W odróżnieniu od push-pull, open-drain nie zwiera wyjścia do napięcia zasilania (3v3). Konfiguracja open-drain nie posiada tranzystora PMOS. Oznacza to, że open-drain ma stany "floating" oraz "GND". Do nadania stanu wysokiego stosuje się pull-up rezystor.

Proponowane rozwiązanie używa mikrokontroler STM32, który ma możliwość ustawienia PINów w konfiguracji open-drain oraz dodania 40kΩ rezystora pull-up widocznego na ilustracji (1).

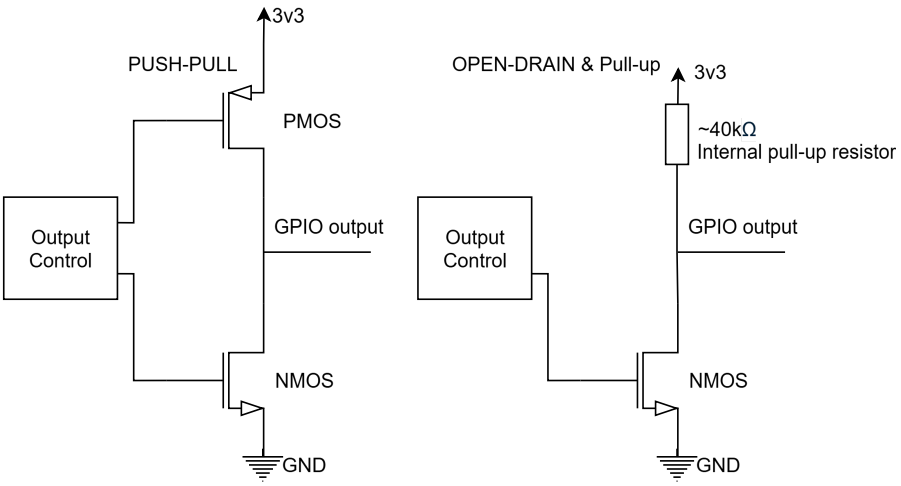


Figure 1: Schemat konfiguracji push-pull i konfiguracji open-drain

3.2 Układ elektryczny

Schemat elektryczny (2) przedstawia propozycje nawiązania komunikacji z TLI4971. Tabela pinów (1) przedstawia konfiguracje pinów w programie STM32CubeIDE. Schemat elektryczny stabilizatora napięcia TC1014 (3).

PIN	Konfiguracja	Opis
PF1 "AOUT"	GPIO output (Open-drain, Pull-up)	PIN odpowiedzialny za wysyłanie bitów do czujnika.
PB11 "ANSWER"	GPIO input	Odbiera dane wysłane przez TL4971. Pin PB11 i PF1 są ze sobą zwarte.
PC8 "POWER"	GPIO output (Open-drain, Pull-up)	Włącza zasilanie na czujnik.

Table 1: Tabela Pinów

4 Protokół komunikacji

4.1 Wstęp

Cała komunikacja bazuje na 16 bitowych słowach/komendach przesyłanej po wspólnym połączeniu z pinem Aout. Podczas przesyłania słowa/komendy do sensora sensor również odsyła 16 bitowy wyraz. Czujnik zawsze odsyła odpowiedź na komendę z opóźnieniem jednego słowa.

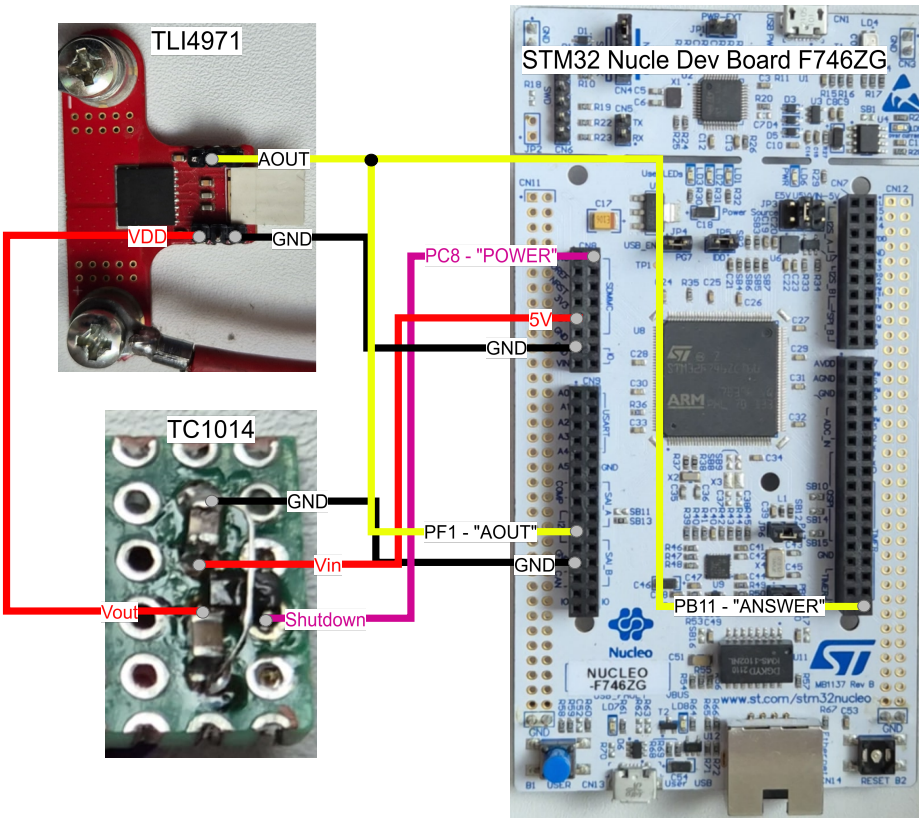


Figure 2: Schemat podłączenia do komunikacji z czujnikiem (brak możliwości programowania EEPROM)

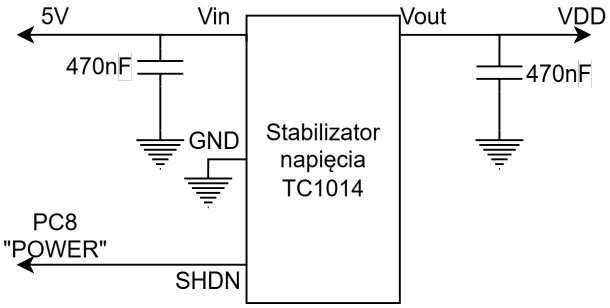
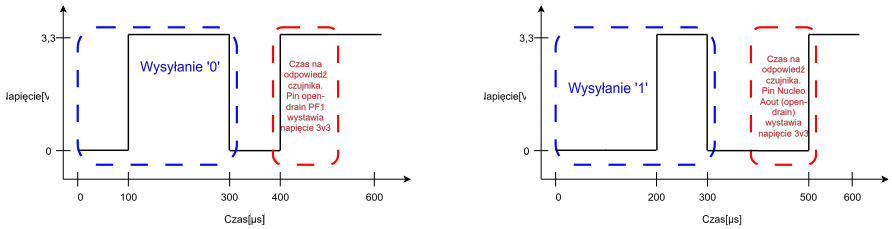


Figure 3: Schemat elektryczny stabilizatora TC1014

4.2 Wysyłanie jednego bita

Żeby wysłać 1 bit z 16 bitowego słowa należy wysłać "ramkę" o sześciu pozycjach. W każdej ramce znajduje się miejsce na wysłanie jednego bita do płytki i odczytania jednego bita z płytki. Przebiegi czasowe ramki przedstawiono na grafikach: (4), Aby dać możliwość sensorowi na przesłanie wartości musimy ustawić płytkę w stan "high-Z" (high impedance), czyli ustawić pin (open-

drain:pull up) "Aout" na stan wysoki, i odczytujemy stan. Stan wysoki (3V3) to logika ’0’, a stan niski (0V) to logika ’1’.



(a) Wysyłanie ’0’, i otrzymanie ’0’

(b) Wysyłanie ’1’, odebranie ’1’

Figure 4: Przebieg wysyłania i odbierania danych

4.3 Komendy

Komenda$_{hex}$	Opis
ABCD	Inicjacja cyfrowego interfejsu.
0xx0	Prośba o przeczytanie pamięci na adresie ’xx’. W miejsce ’xx’ wpisuje się adres danych w formie szesnastkowej.
0400	Prośba o przeczytanie zawartości pamięci o adresie 40_{hex}
04a0	Prośba o przeczytanie zawartości pamięci o adresie $4a_{hex}$
8xx0	Zainicjowanie pisania na adresie ’xx’.
8250 -> 8000	Komendy wyłączające wewnętrzny state machine, żeby nie blokował magistrali danych EEPROMu. Najpierw wysyła się 8250(Inicjowanie pisania na adresie 25_{hex}, potem 8000(16bit słowo)).
8010 -> 0000	Wyłączenie ’failure indictaion’. Wyłączenie tej funckji wymagane jest podczas programowania EEPROM.
8510	Komenda inicjująca zapisywanie na adresie 51_{hex}, następne słowo wysłane przez Nucleo powinno być 16 bitowym wyrazem jaki zapisze się na danym adresie.
FFFF	No operation (NOP) komenda daje okazję czujnikowi na odesłanie odpowiedzi.

Table 2: Tabela komend

Przesyłamy 16-bitowe słowa do czujnika od najmniej znaczącego bita(LSB) do najbardziej znaczącego(MSB). Sensor również przesyła 16 bitowe słowo od LSB do MSB.

Płytka posiada 16 adresów pamięci stałej, na każdym adresie jest 16 bitów informacji (od 40_{hex}, do 51_{hex}). Pierwsze trzy adresy (40_{hex}, 41_{hex}, 42_{hex})

to adresy, którymi można zmieniać ustawienia płytki (zakres pomiaru, tryb działania, ustawienia overcurrent protection (piny OCD), napięcie referencyjne (Pin Vref) i tak dalej). W dokumentacji czujnika TLI4971 w tabeli 9, 10 i 11 znajduje się pełen spis opcji jakie można zmienić w adresach 40_{hex}, 41_{hex} i 42_{hex}. Możemy te rejestry odczytać i nadpisać komendami 8xx0, lecz po ponownym włączeniu płytka powróci do ustawień sprzed zmian.

4.4 Aktywowanie interfejsu cyfrowego

Zanim będzie dostępne wysyłanie i odbieranie danych należy najpierw wejść do interfejsu (ilustracja 5). Aby to zrobić należy od razu po włączeniu zasilania podać stan niski na pin Aout trwający od 100μs do 400μs, potem wystawić na PIN Aout przez ok. 100μs stan wysoki. Następnie należy wysłać słowo $ABCD_{hex}$ zgodnie z zasadami wysyłania ramek podanej w rozdziale 4.2 (ilustracja 6). Należy pamiętać, że wyraz należy wysłać od LSB do MSB. Po wysłaniu słowa ABCD na pinie czujnika Aout powinno pojawić się napięcie 3.3V. Błędne wejście do interfejsu sprawi, że czujnik przejdzie w tryb pracy i na Aout pojawi się napięcie 1,6V. Na ilustracjach (5 i 6) spadające zbocze sygnału "trigger/timing" oznacza początek nowej ramki.

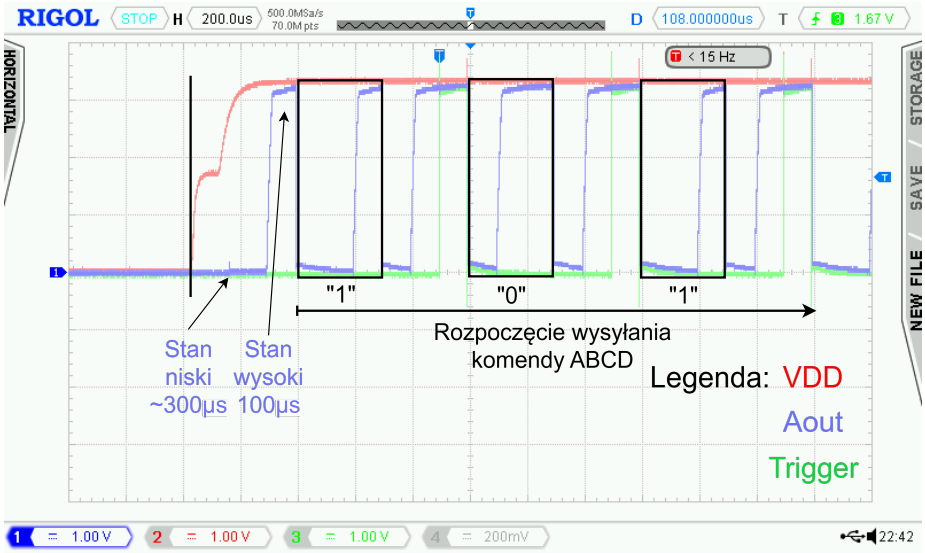


Figure 5: Rozpoczęcie nadawania słowa $ABCD_{hex}$ na czujnik

4.5 Zmienianie pamięci trwałej

4.5.1 Sekwencja zapisania pamięci trwałej

W celu zmiany ustawień w pamięci trwałej należy wykonać następującą sekwencję: wysłanie komendy zapisu, wysłanie danych, wysłanie komendy EEPROM, programowanie "1", podanie napięcia programującego, wysłanie komendy EEPROM, odświeżenie EEPROM, przerwa, wysłanie komendy zapisu, wysłanie danych, wysłanie komendy EEPROM, programowanie "0", podanie napięcia programującego, wysłanie komendy EEPROM i odświeżenie EEPROM.

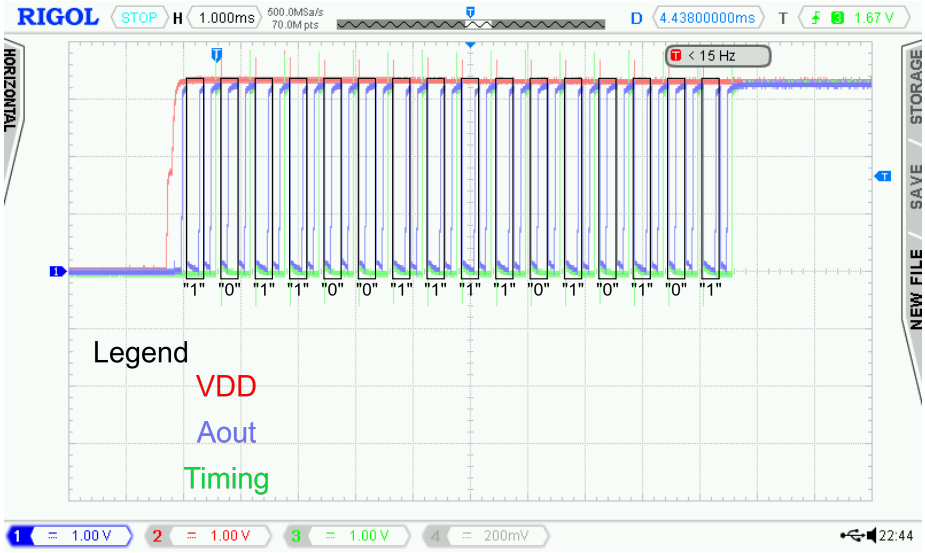


Figure 6: Pełen przebieg inicjacji interfejsu z czujnikiem.

4.5.2 Ustawienia czujnika

Ustawienia czujnika zmienia się wysyłając odpowiednie wartości na 64, 65 i 66 słowa czujnika, które odpowiadają za pamięć EEPROM. Pierwsze 4 bity 64 słowa odpowaiadają za zakres pomiarowy, następne dwa za tryb operacyjny. Następne bity odpowiadają za czas na naprawę błędu OCD1 i OCD2 oraz poziom na którym wykrywane jest przeciążenie. 65 słowo odpowiada za ustawienie progów OCD1 i OCD2. Ostatnie słowo zawiera w sobie wartość CRC oraz ustawienia kolejno: wartości V_{ref}, "spoczynkowego" napięcia dla trybu półróżnicowego i wyłączenia wskaźnika przeciążenia dla OCD2. Szczegółowe opisy poszczególnych bitów przedstawiono w dokumentacji programowania czujnika.

4.5.3 Obliczanie CRC

Do poprawnego przeprowadzenia zmiany pamięci EEPROM, wykorzystywany jest algorytm pozwalający wyliczyć wartość CRC - cyklicznej kontroli nadmiarowej (ang. Cyclic Redundancy Check). Oblicza się ją na podstawie algorytmu zamieszczonego w dokumentacji. Do obliczenia wykorzystywane są niezmienne, odczytane wartości od 4 do 18 słowa pamięci trwałej (obszar tylko do odczytu) i zapisywane wartości pierwszych trzech słów. Algorytm stosuje się dla docelowych wartości 1-3 słowa EEPROM, tym samym otrzymując ostateczną wartość drugiego bajtu trzeciego słowa. Niepoprawne obliczenie CRC skutkuje odcięciem zasilania płytki.

4.5.4 Zastosowanie

W programie zaimplementowano funkcję obliczającą CRC na podstawie fragmentu kodu zamieszczonego w dokumentajcji. Następnie odnaleziono wartość 64 słowa odpowiadającą zakresowi $\pm 50A$. Wybraną wartość zamieszczono w

programie, jednocześnie pozostawiając domyślne wartości na pozostałych bitach. Z wykorzystaniem sekwencji zapisu pamięci trwałej, a także algorytmu obliczającego CRC, skutecznie zmieniono wartość 64 słowa. Przetestowanie faktycznego zapisu do pamięci trwałej, nie było wykonalne ze względu na brak możliwości podania napięcia programującego.

5 Program

5.1 Wstęp

Zrealizowany program opiera się na przerwaniu czasowym licznika TIM3 (co 100μs). Po inicjalizacji program sczytuje wszystkie rejestry EEPROMu, oblicza wartość CRC na podstawie zawartości EEPROMu, wpisuje nowe rejsetry EEPROMu, zapisuje je i znowu sczytuje pamięć EEPROMu. Proces ten opisany jest w schemacie działania programu (rysunek 7).

5.2 Działanie programu

Zaproponowany program programujący EEPROM (rysunek: 7) działa w następujących krokach:

- **case 0:** Porgram oczekuje na wciśnięcie przycisku USR_BTN, aby zainicjować program. Ten case również resetuje odpowiednie liczniki.
- **case 1:** Program włącza zasilanie pinem PC8 - "POWER" (schemat podłączenia: 2), ustawia na PIN PF1 "AOUT" stan niski przez 300μs i stan wysoki na 100μs.
- **case 2:** Program następnie wysyła ramkę ABCD (przebiegi na rysunku: 5 i 6) inicjując interfejs cyfrowy. Program nie odbiera tego co czujnik wysyła, ponieważ nie jest to istotne.
- **case 3:** Program wyłącza wewnętrzną maszynę stanów czujnika, żeby nie blokował głównej magistrali EEPROM (tabela 2). Potem wyłącza 'failure indication'. Następnie każe czujnikowi przesłać całą zawartość EEPROMu. Oblicza na podstawie wyczytanego EEPROMu nową wartość CRC z uwzględnieniem zamierzonych zmian.
- **case 4:** Program za pomocą specjalnej sekwencji słów (zawartej w tablicy 'ProgrammingSeq') przesyła i zapisuje linia po linii rejestry EEPROMu.

```
uint16_t ProgrammingSeq[12] = {0x8400, 0x0, 0x83e0, 0x024f,
0x83e0, 0x024c, 0x8400, 0x0, 0x83e0, 0x024e, 0x83e0, 0x024c};
```

  - Pierwsze dwie komendy (oraz koemdny siódme i ósme) "0x8400 i 0x0" to zapisywanie na adresie 40_{hex} wartości "0". Zarówno adres jak i wartość ulegają zmianie podczas wykonywania się kodu:

```
ProgrammingSeq[0] += 0x10;
ProgrammingSeq[1] = WriteWords[sequence_counter];
ProgrammingSeq[6] += 0x10;
ProgrammingSeq[7] = WriteWords[sequence_counter];
```

  - 0x83e0 i 0x024f: na adresie $3e_{hex}$ zapisz wartość 0x024f. Taka komenda sprawia, że czujnik programuje jedynki z zapisanej wartości do stałego rejestru EEPROM.
  - Po wysłaniu zapisania samych jedynek program inicjuje 'case 5', który przez 30ms podaje V_{prog} na pin OCD2.
  - 0x83e0 0x024c to komenda 'odswież'.
  - Po wysłaniu komendy 'odswież' program wchodzi w 'case 6', który odczekuje 100μs, według zaleceń dokumentacji TLI4971.
  - W następnych krokach program wysyła te same komendy tylko zamiast komend "0x83e0 -> 0x024f" wysyła "0x83e0 -> 0x024e". 0x024e to komenda zapisująca wszystkie zera.

- **case 5:** Po zapisaniu zmian program ponownie sczytuje wartości rejestrów ze wszystkich adresów EEPROMu. Jeśli program zadziałał poprawnie to zmiany zostały zapisane na stałe, a EEPROM po ponownym połączeniu będzie działał na podanych ustawieniach.

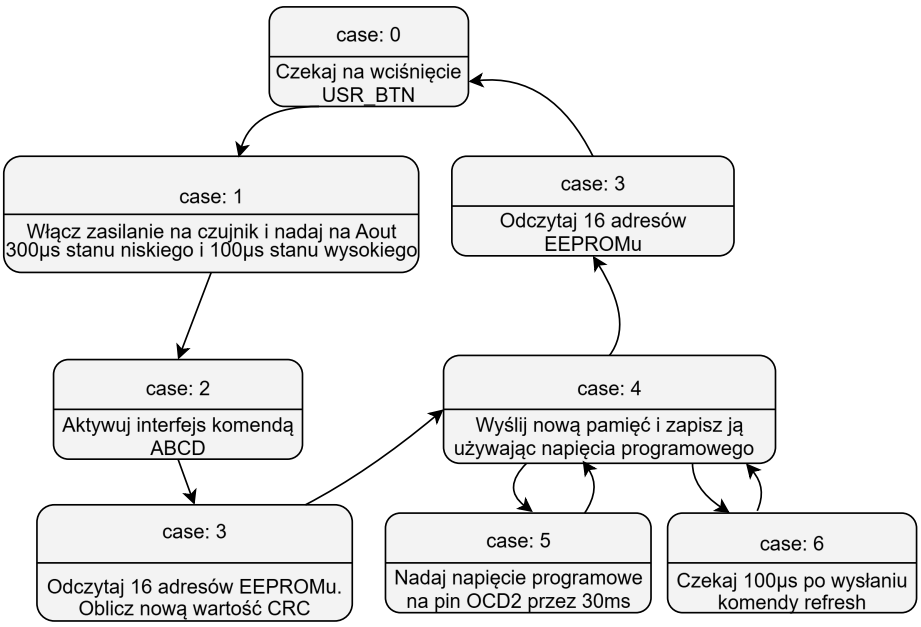


Figure 7: Schemat blokowy programu do programowania pamięci EEPROM czujnika

5.3 Tryby pracy programu

Program zamiast programowania EEPROMu może również ograniczyć się jedynie komunikacji z czujnikiem. Niniejszy podrozdział przedstawi dodatkowe funkcje programu i jak z nich korzystać.

1. **Pojedyncze sczytywanie rejestrów EEPROMu:** Program można ograniczyć do jedynie sczytania wszystkich rejestrów EEPROMu w celu weryfikacji działania programu. W 'case 3' należy zmienić wartość 'checkmark' na '0' (fragment kodu przedstawiono poniżej).

```
// Pojedyncze odczytanie rejstrów EEPROMu:
else{
    reg_selector = 1;
    crc_value = crcCalc(EPROMreg[0], 18);
    WriteWords[2] += crc_value;
    // checkmark = 4; // <- comment out this line for single
    ↪  EEPROM read
    checkmark = 0; // <- instert this line for single EEPROM
    ↪  read
}
```

2. **Pisanie swobodne:** w trybie pisania swobodnego można dowolnie wysyłać i odczytywać słowa. Żeby ustawić program na tryb swobodnego pisania należy w 'case 2' zmienić przypisywaną wartość zmiennej 'checkmark' z '3' na wartość '7'.

```
if(bit_counter >= 6){
    HAL_GPIO_WritePin(TRIGGER_GPIO_Port, TRIGGER_Pin,1);
    bit_counter = 0;
    picker = picker << 1;
    if(!picker){
        picker = 1;
        //checkmark = 3; // Comment out this line to write
        ↪  custom message and read the answer form TLI4971
        checkmark = 7; // Uncomment this line, to write
        ↪  custom message and read the answer from TLI4971
    }
}
```

Do trybu swobodnego pisania dodano również tablice 'AnswerSchematic', gdzie jedynki to pozycje, w których odpowiedź jest zapisywana, a zera tam gdzie odpowiedź czujnika jest pomijana. W taki sposób można czytelniej odczytywać informacje podawane przez czujnik.

```
const uint16_t CustomMsg[4] = {0x8250, 0x8000, 0x04e0,
↪  0xffff}
const uint8_t AnswerSchematic[4] = {0, 0, 0, 1};
```

W powyższym przykładzie odpowiedź będzie zapisana jedynie podczas gdy płytka nucleo wysyła komendę 0xffff.

CustomMsg	const uint16_t [4]	[4]
CustomMsg[0]	const uint16_t	0x8250
CustomMsg[1]	const uint16_t	0x8000
CustomMsg[2]	const uint16_t	0x180
CustomMsg[3]	const uint16_t	0xffff
AnswerCustomMsg	uint16_t [4]	[4]
AnswerCustomMsg[0]	uint16_t	0x584
AnswerCustomMsg[1]	uint16_t	0x0
AnswerCustomMsg[2]	uint16_t	0x0
AnswerCustomMsg[3]	uint16_t	0x0
Add new expression		

Figure 8: Widok z sesji Debug (zakładka live expressions)

3. **Przykład komunikacji swobodnej** Przykładem komunikacji swobodnej jest sczytanie temperatury czujnika (pamiętając, że czujnik podaje temperaturą zawyżoną o około 5°C). Czujnik zapisuje zmierzoną temperature na adresie 18_{hex}. Zdefiniowana poniżej tablica 'CustomMsg' najpierw wyłącza wewnętrzną maszynę stanów płytki, a następnie sczytuje adres czujnika 18_{hex}.

```
const uint16_t CustomMsg[4] = {0x8250, 0x8000, 0x0180,
↪  0xffff}
const uint8_t AnswerSchematic[4] = {0, 0, 0, 1};
```

Wartość temperatury to wartość 0x584 (ilustracja 8). Równania (1) i (2) pokazują jak przejść z wartości wysłanej przez czujnik na temperaturę otoczenia w [°C]. Jeśli temperatura otrzymana zgadza się z przybliżoną temperaturą otoczenia to można uznać, że proces wysyłania i odczytywania z czujnika danych jest porawny.

$$T[°C] = \frac{Value_{10} - 2048}{16} + 65 \tag{1}$$

$$25,25[°C] = \frac{1412 - 2048}{16} + 65 \tag{2}$$

6 Problem ze stabilizatorem napięcia

Ta sekcja opisze problem ze stabilizatorem napięcia, napotkany podczas testowania. Jeżeli po wysłaniu komendy 0xABCD czujnik będzie wysyłał ∼1.6V, zamiast 3.3V to znaczy, że czujnik nie zainicjował komunikacji cyfrowej. Powodów może być wiele, na przykład przerwy w przewodach, ale może też być to problem spowodowany stabilizatorem napięcia. Ten rozdział skupi się na tym właśnie problemie.

Wiadome jest, że komendę 0xABCD należy poprzedzić pojedynczym krótkim impulsem 'low' (na 100µs - 400µs), i drugim impulsem 'high' na ∼100µs. Kod do wysyłania tych dwóch pierwszych impulsów znajduje się poniżej:

```
const uint8_t TIFEN_Matrix[6] = {0,0,0,1,1,1};
```

```
case 1: //Powering up the sensor
        HAL_GPIO_WritePin(AOUT_GPIO_Port, AOUT_Pin,
        ↪  TIFEN_Matrix[bit_counter]); //setting 0 to Aout.
        HAL_GPIO_WritePin(POWER_GPIO_Port, POWER_Pin, SET);
        ↪  //Turning on voltage stabilizer TC1014
        bit_counter ++ ;
        if(bit_counter >= 4){
          bit_counter = 0;
          checkmark = 2;
        }
        break;
```

Jednakże może zdarzyć się tak, że stabilizator nie zdąży wystarczająco szybko zasilić czujnika, co spowoduje opóźnieniem w reakcji czujnika na sygnały wysyłane przez płytkę, i impuls 'low' będzie krótszy niż wymagane 100µs.

Rozwiązanie: Należy wtedy zmodyfikować kod (modyfikując macierz : TIFEN_Matrix[6] = 0,0,0,0,1,1, oraz warunek if(bit_counter >= 5)). Jeśli czujnik po modyfikacji kodu nadal nie zainicjuje komunikacji cyfrowej, wtedy należy podpiąć oscyloskop do komunikacji. Jedną sondę do Aout, a drugą do pinu zasilania płytki. Następnie ocenić czy ramy czasowe zostały uzyskane. Jeśli ramy czasowe są zgodne to problem tkwi gdzie indziej. Propozycje:

- Sprawdzić czy sygnał T_{ifen}(interface enable) jest podobny z przebiegiem pokazanym na rysunku (5).
- Sprawdzić czy sygnał 0xABCD jest poprawnie wysyłany zgodnie z rysunkiem (6).
- Porównać Pin VDD (pin zasilania czujnika) z pinem PF1 "Aout". Sprawdzić czy stabilizator osiąga 3v3 w przeciągu 200µs lub mniej, tak jak na rysunku (5).